LegalEase

AI-Powered Legal Document Generator

Viva Questions & Answers

Prepared in correct project order from the uploaded LegalEase documentation.

1. Project Introduction

1. What is LegalEase?

Answer: LegalEase is an AI-powered legal document generator for creating structured documents such as contracts, NDAs, lease agreements, and employment documents.

2. What is the main objective?

Answer: To generate customizable, editable, and formatted legal documents from user inputs.

3. Who can use it?

Answer: Entrepreneurs, professionals, and individuals.

4. What problem does it address?

Answer: It simplifies the creation of structured legal documents through an AI-assisted workflow.

5. What outputs are supported?

Answer: TXT, DOCX, and PDF.

2. Requirements and Workflow

6. What are the prerequisites?

Answer: FastAPI, Gemini API, Streamlit, and Python knowledge, plus the required Python packages.

7. Which libraries are listed?

Answer: FastAPI, Uvicorn, Streamlit, python-docx, FPDF, Pillow, requests, google-generativeai, and python-dotenv.

8. What is the project workflow?

Answer: Model selection and architecture, core functionality, API logic, frontend, deployment, and final testing.

9. What inputs are required?

Answer: Document type, parties, terms and conditions, and effective date.

10. Why is .env used?

Answer: The documentation specifies it for storing the Gemini API key.

3. AI Model and Architecture

11. Which AI model is selected?

Answer: Gemini 1.5 Pro.

12. Why is Gemini used?

Answer: The project requires structured legal text generation from multiple user inputs.

13. What are the architecture components?

Answer: Frontend, backend, AI core, and formatting modules.

14. What is Streamlit's role?

Answer: User input, preview, editing, and downloads.

15. What is FastAPI's role?

Answer: It receives requests, validates inputs, invokes the AI generator, and returns content.

4. Backend and API

16. What is main.py responsible for?

Answer: Initializing FastAPI, providing the root endpoint, and including routes.

17. What is routes.py responsible for?

Answer: Document-generation API logic.

18. What is DocumentRequest?

Answer: A Pydantic model used to validate incoming document-generation data.

19. What is /generate?

Answer: A POST endpoint that accepts document inputs and returns generated legal content.

20. How does the backend call AI?

Answer: GeminiDocumentGenerator constructs a structured prompt and sends it to Gemini.

5. Frontend

21. Why use Streamlit?

Answer: It provides a simple web interface for entering inputs and displaying generated documents.

22. What input fields are provided?

Answer: Document type, parties, terms, and effective date.

23. How are terms entered?

Answer: The documentation specifies semicolon-separated terms.

24. What happens after Generate Document?

Answer: The frontend sends a POST request to the FastAPI backend.

25. Can the document be edited?

Answer: Yes, through an editable text area.

6. Formatting and Export

26. What is sanitize_text?

Answer: It removes special characters and typographic quotes for clean formatting.

27. What does format_docx do?

Answer: It creates a formatted Word document using python-docx.

28. What does format_pdf do?

Answer: It creates a formatted PDF using FPDF with headings, branding, terms, and footer.

29. What does format_html_preview do?

Answer: It creates styled HTML content for Streamlit preview.

30. What formats can users download?

Answer: TXT, DOCX, and PDF.

7. Deployment and Testing

31. How is it deployed locally?

Answer: Create a virtual environment, install requirements, configure .env, start FastAPI, and start Streamlit.

32. How are dependencies installed?

Answer: Using pip install -r requirements.txt.

33. What should be tested?

Answer: Inputs, generation, preview, editing, downloads, logo/footer layout, bullet formatting, and terms.

34. Which server runs FastAPI?

Answer: Uvicorn.

35. Which frontend is started?

Answer: The Streamlit app, referenced as app.py in the documentation.

8. Practical User Flow

36. Give document type examples.

Answer: Agreement, Contract, NDA, Lease Agreement, and Employment Offer Letter.

37. Give a parties example.

Answer: Jane Doe (Service Provider), TechNova Inc. (Client).

38. Give terms examples.

Answer: Payment within 30 days, delivery deadline, confidentiality, and termination notice.

39. What is the effective date?

Answer: The date from which the agreement is intended to become legally valid.

40. What happens after generation?

Answer: The user can preview, edit, and download the document.

9. Conclusion and Future Scope

41. What is the main technical idea?

Answer: Streamlit frontend + FastAPI backend + Gemini AI generation + document-formatting utilities.

42. What are the key benefits?

Answer: Accessible generation, editable previews, structured documents, and multiple export formats.

43. What future enhancements are mentioned?

Answer: Deeper contract analysis, legal database integration, and personalized recommendations.

44. What is the final goal?

Answer: To make legal document creation and understanding more accessible through AI.

45. What should be remembered for viva?

Answer: Know the architecture, inputs, /generate API, Gemini integration, Streamlit UI, export formats, deployment, and testing.

10. Quick Viva Revision

46. Explain LegalEase in 30 seconds.

Answer: LegalEase is an AI-powered legal document generator using Streamlit, FastAPI, and Gemini. Users enter document type, parties, terms, and date; the system generates content that can be previewed, edited, and exported as TXT, DOCX, or PDF.

47. Why FastAPI?

Answer: It provides the backend API layer for processing document-generation requests.

48. Why Streamlit?

Answer: It provides the user-facing interface.

49. Why Gemini?

Answer: It generates structured legal content from the supplied inputs.

50. What is the complete flow?

Answer: User input → Streamlit → FastAPI /generate → GeminiDocumentGenerator → Gemini → generated text → preview/edit → TXT/DOCX/PDF.